DSIC
DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y COMPUTACIÓN

Escuela Técnica Superior de Ingeniería Informática etsinf

Bases de Datos

Grado en Ciencias de Datos

Proyecto:

Traslado de sede del DSIC

Miquel Àngel Crespí García

Raúl Benítez Casanova

Pedro Luque Gimeno

Jaime Planells Tornero

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

ÍNDICE

1 ENUNCIADO DEL PROBLEMA: G34 TRASLADO DE SEDE DEL DSIC 3

2 TAREA 1: ANÁLISIS Y DISEÑO CONCEPTUAL 4
2.1 DIAGRAMA DE CLASES 4
2.2 RESTRICCIONES DE INTEGRIDAD QUE NO SE HAN PODIDO EXPRESAR 5
2.3 REQUISITOS DE PROCESO Y/O CONSULTA 5

3 TAREA 2: DISEÑO LOGICO 6
3.1 ESQUEMA RELACIONAL 6
3.2 RESTRICCIONES DE INTEGRIDAD QUE NO SE HAN PODIDO EXPRESAR 8
3.3 NORMALIZACIÓN 9
3.4 DIRECTRICES DE RESTAURACION DE LA INTEGRIDAD REFERENCIAL 11
3.5 TRANSACCIONES 12

4 TAREA 3: CREACIÓN Y CARGA DE LAS BASES DE DATOS EN ORACLE 13
4.1 INSTRUCCIONES DE CREACIÓN DE LA BASE DE DATOS 13
4.1.1 Creación de la tabla EDIFICIO: 13
4.1.2 Creación de la tabla DESPACHO: 13
4.1.3 Creación de la tabla DESPACHO_NUEVO: 13
4.1.4 Creación de la tabla DESPACHO_VIEJO: 14
4.1.5 Creación de la tabla PROFESOR: 14
4.1.6 Creación de la tabla CONTACTO: 14
4.1.7 Creación de la tabla HOJA_TRASLADO: 14
4.1.8 Creación de la tabla BULTO: 15
4.1.9 Creación de la tabla RECLAMACIÓN: 15
4.1.10 Creación de la tabla SE_DAÑA: 15

4.2 DIAGRAMA GENERADO POR ORACLE CON EL DATA MODELER 16
4.3 CARGA DE LA BASE DE DATOS 16
4.3.1 Funciones de carga 16
4.3.1.2 Función crear_situación 17
4.3.1.3 Función crear_observaciones 17
4.3.1.4 Función crear_contenido 18
4.3.1.5 Función crear_categoria_laboral 18
4.3.1.6 Función contenido_frágil 18
4.3.2 Disparadores 19
4.3.2.1 Disparador del_desp_nuevo_pend 19
4.3.2.2 Disparador ins_desp_nuevo_pend 19
4.3.2.3 Disparador del_desp_pend 20
4.3.2.4 Disparador ins_desp _pend 20
4.3.3 Carga de tablas 20
4.3.3.1 Carga de la tabla EDIFICIO 20
4.3.3.2 Carga de la tabla DESPACHO 21
4.3.3.3 Carga de la tabla DESPACHO_VIEJO 21
4.3.3.4 Carga de la tabla DESPACHO_NUEVO 22
4.3.3.5 Carga de la tabla PROFESOR 22
4.3.3.6 Carga de la tabla HOJA_TRASLADO 23
4.3.3.7 Carga de la tabla BULTO 24
4.3.3.8 Carga de la tabla RECLAMACIÓN 25
4.3.3.9 Carga de la tabla SE_DAÑA 26
4.3.3.10 Carga de la tabla CONTACTO 26

5. TAREA 4: INFORMES Y PROGRAMACIÓN 28
5.1 INFORMES 28
5.1.1 INFORME 1 28
5.1.2 INFORME 2 29
5.1.3 INFORME 3 29
5.1.4 INFORME 4 30
5.1.5 INFORME 5 31

5.1.6 INFORME 6 31
5.2 PROGRAMACION 32
5.2.1 FUNCIONES....32
5.2.1.1 FUNCION RECLAMACION 32
5.2.1.2 FUNCION BULTO CORRESPONDIENTE....32
5.2.1.3 FUNCION DESPACHOS NUEVOS PENDIENTES 33
5.2.1.4 FUNCION DESPACHOS VIEJOS PENDIENTES....33
5.2.2 PROCEDIMIENTOS 33
5.2.2.1 PROCEDIMIENTO ELIMINAR EDIFICIOS 33
5.2.2.2 PROCEDIMIENTO AÑADIR DESPACHOS SUR 34
5.2.2.3 PROCEDIMIENTO HOJA RECLAMACION 35
5.2.2.4 PROCEDIMIENTO PROFESORES YA TRASPASADOS 36
5.2.3 DISPARADORES....36
5.2.3.1 DISPARADOR BULTO DAÑADO....36
5.2.3.2 DISPARADOR DESPACHO NUEVO/VIEJO 37
5.2.3.3 DISPARADOR DESPACHO NUEVO/VIEJO 37
5.2.3.4 DISPARADOR FECHA RECLAMACION....38
5.2.3.5 DISPARADOR FECHA PREVISTA Y REALIZADA....38
5.2.3.6 DISPARADOR RESTAR VIEJOS PENDIENTES 39

1 ENUNCIADO DEL PROBLEMA: G34 TRASLADO DE SEDE DEL DSIC

Con el fin de optimizar el cambio a su nueva sede departamental ha decidido diseñar una base de datos para facilitar la mudanza. El departamento dispone de despachos que se identifican por un código que es único dentro de cada edificio de los que dispone. Los edificios del departamento se identifican por un código único dentro de la upv y se conoce su situación y su superficie en metros cuadrados. De los despachos interesa saber la orientación (n, s, e, w) y también cuántos despachos están aún pendientes de traslado.

La mudanza se realiza de los despachos viejos a los nuevos. Cada profesor, del cual se conoce su dni, que lo identifica, su nombre, uno o varios teléfonos, su categoría laboral, y el despacho viejo que ocupaba, tiene asignado un despacho nuevo. Los despachos viejos, generalmente, estaban compartidos por más de un profesor y todos estaban ocupados. Los nuevos, por el contrario, son despachos individuales y hay algunos, que por el momento, pueden quedar libres.

Se ha elaborado una planificación en la que se indica el día previsto en que se realizará el traslado de un despacho viejo a uno nuevo. Todos los profesores de un despacho se trasladan el mismo día. Como puede haber retrasos, en el momento en que se realice realmente el traslado se debe almacenar el día exacto del mismo.

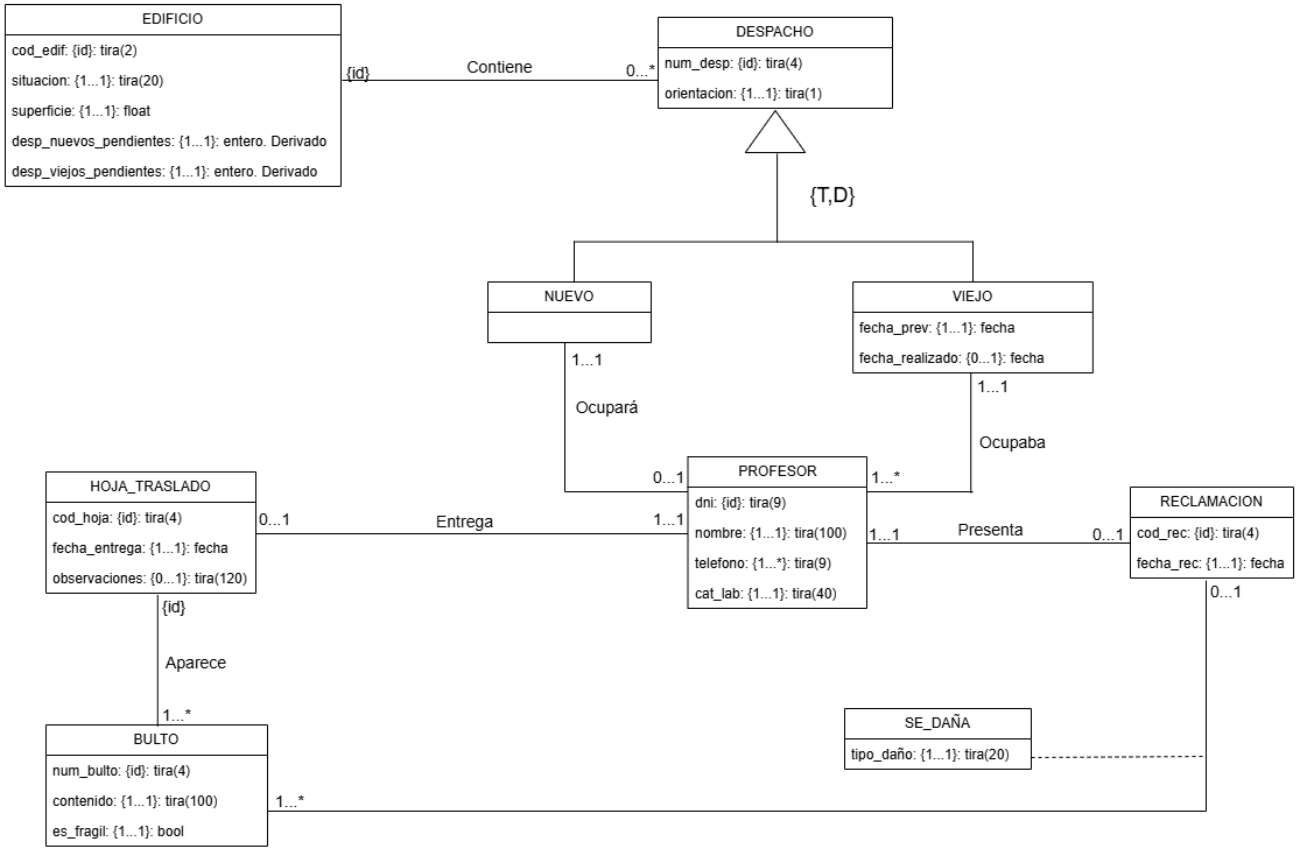
Antes del traslado, cada profesor debe presentar en la secretaría un documento llamado “hoja de traslado” en el que debe figurar la fecha de entrega de la hoja, el profesor que la presenta y las observaciones que considere oportunas para la empresa de mudanzas. Además, dicha hoja debe incluir una lista de los bultos que se van a trasladar. Cada bulto se identifica por un número que es único dentro de cada hoja y se debe indicar claramente su contenido y si es frágil o no.

Es posible que algún bulto resulte dañado en el traslado, si esto ocurre, un profesor puede presentar en la secretaría una reclamación (como mucho) en la que debe especificar cuáles de sus bultos han sufrido daños y de qué tipo (pérdida, rotura, …). A cada reclamación, se le asignará un código único en el momento en que se entrega y se guardará la fecha en que se presentó.

2 TAREA 1: ANÁLISIS Y DISEÑO CONCEPTUAL

Tarea	Contenido	Puntos	Trabajo a realizar	Contenido de la entrega
1	Análisis del SI Diseño Conceptual	1,5 puntos	• Diseño Conceptual comprobando que el diagrama de clases obtenido satisface todas las reglas del "Test de corrección de un diagrama de clases". • El diagrama de clases debe contener al menos: ▪ una especialización ▪ una clase débil ▪ una clase asociación ▪ un atributo multivaluado ▪ un atributo fecha ▪ un atributo booleano ▪ un atributo derivado	Un solo documento pdf que incluya: • Enunciado del problema. • Diagrama de clases LEGIBLE. • Lista de restricciones de integridad que no se hayan podido expresar en el diagrama (aparte o en el propio diagrama). • Lista de requisitos de proceso y/o consulta.

2.1 DIAGRAMA DE CLASES



2.2 RESTRICCIONES DE INTEGRIDAD QUE NO SE HAN PODIDO EXPRESAR

1. El atributo *orientación* de la clase *Edificio* toma un valor entre {N,S,W,E}.
2. El atributo *fecha_prev* de la subclase *Viejo* deber ser mayor al atributo *fecha_entrega* de la clase *Hoja_Traslado.*
3. El atributo *fecha_realizado* de la subclase *Viejo* deber ser mayor al atributo *fecha_entrega* de la clase *Hoja_Traslado.*
4. El atributo *desp_nuevos_pendientes* de *Edificio* es un atributo derivado que se mantendrá automáticamente por el sistema y su valor será en todo momento la cantidad de despachos nuevos que están pendientes de traslado de cada edificio.
5. El atributo *desp_viejos_pendientes* de *Edificio* es un atributo derivado que se mantendrá automáticamente por el sistema y su valor será en todo momento la cantidad de despachos viejos que están pendientes de traslado de cada edificio, es decir todos aquellos despachos que el atributo *fecha_realizado* sea nulo.
6. El atributo *fecha_rec* de la clase *Reclamación* debe ser mayor al atributo *fecha_realizado* de la subclase *Viejo*.

2.3 REQUISITOS DE PROCESO Y/O CONSULTA

En este apartado aparecen los requisitos tanto de proceso como de consulta

Requisitos de consulta:

- Consultar el código de hoja, número de bulto y el contenido de los bultos frágiles.
- Consultar el DNI y nombre de los profesores que ya han realizado el traslado.
- Consultar el código y las observaciones de las hojas de reclamaciones, ordenadas por la fecha de entrega.
- Consultar todos los bultos dañados por rotura.
- Consultar el código y situación de los edificios con más de 150 metros cuadrados.
- Consultas el código de los despachos nuevos que están orientados al norte.

Requisitos de procesos:

- Eliminar los edificios que no tengan ningún despacho.
- Dar de alta un nuevo profesor.
- Calcular los despachos nuevos pendientes de traslado.
- Asignar la fecha de realización de un traslado.
- Presentar una reclamación por la pérdida de un bulto.
- Añadir un despacho nuevo.

3 TAREA 2: DISEÑO LOGICO

Tarea	Contenido	Puntos	• Trabajo a realizar	Contenido de la entrega
2	Diseño Lógico	0,75 puntos	• Diseño lógico. • Normalización.	Un solo documento pdf que incluya: • Resultado de la fase anterior con las correcciones hechas por el profesor. • Esquema relacional textual (con la notación académica vista en clase). • Lista de restricciones de integridad que no se hayan podido representar en el esquema lógico. • Reflexión sobre la Normalización. • Reflexión sobre las directrices de restauración de la integridad referencial. • Explicación de las transacciones para resolver los requisitos de la fase anterior.

3.1 ESQUEMA RELACIONAL

Edificio (cod_edif: tira(2), situación: tira(20), superficie: float, desp_nuevos_pendientes: entero, desp_viejos_pendientes: entero)

CP: {cod_edif}

VNN: {situación, superficie, desp_nuevos_pendientes, desp_viejos_pendientes}

Despacho (cod_edif: tira(2), num_desp: tira(4), orientación: tira(1))

CP: {cod_edif, num_desp}

VNN: {orientación}

COMP: (orientación =’N’ O orientación =’S’ O orientación =’E’ O orientación =’W’)

Caj: {cod_edif} Referencia Edificio(cod_edif)

Borrado restrictivo

Modificación en cascada

Despacho_nuevo (cod_edif: tira(2), num_desp: tira(4))

CP: {cod_edif, num_desp}

Caj: {cod_edif, num_desp} Referencia Despacho(cod_edif, num_desp)

Borrado en cascada

Modificación en cascada

Despacho_viejo (cod_edif: tira(2), num_desp: tira(4), fecha_prev: fecha, fecha_real: fecha)

CP: {cod_edif, num_desp}

VNN: {fecha_prev}

Caj: {cod_edif, num_desp} Referencia Despacho(cod_edif, num_desp)

Borrado en cascada

Modificación en cascada

Profesor (dni: tira(9), nombre: tira(20), cat_lab: tira(10), cod_edif_viejo: tira(2), num_desp_viejo: tira(4), cod_edif_nuevo: tira(2), num_desp_nuevo: tira(4))

CP: {dni}

VNN:{nombre, cat_lab, cod_edif_viejo, num_desp_viejo, cod_edif_nuevo, num_desp_nuevo}

Único: {cod_edif_nuevo, num_desp_nuevo}

Caj: {cod_edif_viejo, num_desp_viejo} Referencia Despacho_viejo(cod_edif,num_desp}

Borrado restrictivo

Modificación en cascada

{cod_edif_nuevo, num_desp_nuevo} Referencia Despacho_nuevo (cod_edif,num_desp}

Borrado restrictivo

Modificación en cascada

Contacto (dni:tira(9), teléfono: tira(9))

CP: {dni,telefono}

Caj: {dni} Referencia Profesor(dni)

Borrado en cascada

Modificación en cascada

Hoja_Traslado (cod_hoja: tira(4), fecha_entrega: fecha, observaciones: tira(100), dni: tira(9))

CP: {cod_hoja}

VNN: {fecha_entrega, dni}

Unico: {dni} Caj: {dni} Referencia Profesor(dni)

Borrado en cascada

Modificación en cascada

Bulto (cod_hoja: tira(4), num_bulto: tira(4), contenido: tira(100), es_fragil: booleano)

CP: {cod_hoja, num_bulto}

VNN: {contenido,es_fragil}

Caj: {cod_hoja} Referencia Hoja_Traslado(cod_hoja)

Borrado en cascada

Modificación en cascada

Reclamación (cod_rec: tira(4), fecha_rec: fecha, dni: tira(9))

CP: {cod_rec}

VNN:{fecha_rec, dni}

Único: {dni}

Caj: {dni} Referencia Profesor(dni)

Borrado restrictivo

Modificación en cascada

Se_daña (cod_hoja: tira(4), num_bulto, tipo_daño: tira(20), cod_rec: tira(4))

CP: {cod_hoja, num_bulto}

VNN: {tipo_daño, cod_rec}

Caj: {cod_hoja, num_bulto} Referencia Bulto(cod_hoja, num_bulto)

Borrado en cascada

Modificación en cascada

{cod_rec} Referencia Reclamación(cod_rec)

Borrado en cascada

Modificación en cascada

3.2 RESTRICCIONES DE INTEGRIDAD QUE NO SE HAN PODIDO EXPRESAR

1) Todo despacho es o nuevo o viejo, pero no ambas cosas a la vez.

2) Todo profesor tiene al menos un teléfono.

3) El cálculo del número de despachos nuevos pendientes se realiza contando el número de profesores que en su despacho viejo la fecha_real sea nulo.

4) El cálculo del número de despachos viejos pendientes se realiza contando cuantas tuplas de la tabla Despacho_viejo tienen el atributo fecha_realizado con un valor nulo.

5) Todo despacho viejo está ocupado.

6) El atributo fecha_prev de la clase Despacho_viejo deber ser mayor al atributo fecha_entrega de la clase Hoja_Traslado.

7) El atributo fecha_realizado de la clase Despacho_viejo deber ser mayor al atributo fecha_entrega de la clase Hoja_Traslado.

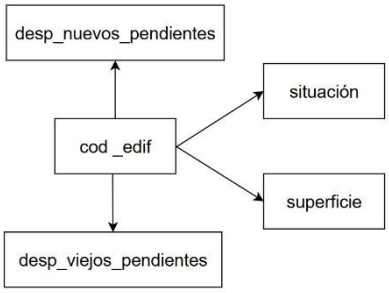
8) El atributo fecha_rec de la clase Reclamación debe ser mayor al atributo fecha_realizado de la clase Despacho_viejo.

9) Un profesor solo puede incluir en una reclamación aquellos bultos que pertenezcan a una de hoja de traslado entregada por él mismo.

10) Toda hoja de reclamación debe estar al menos una vez en Se_daña.

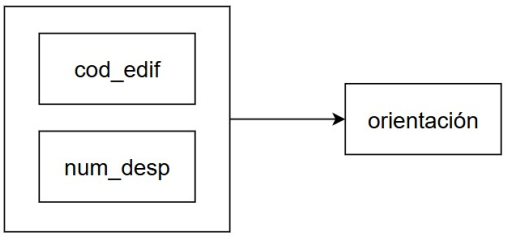
3.3 NORMALIZACIÓN

Relación *Edificio*: está en 3FN.



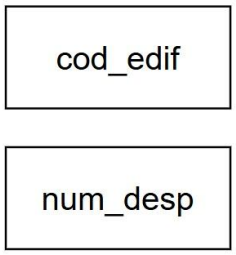
- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (situación, superficie, desp_nuevos_pendientes, desp_viejos_pendientes) dependen de forma completa de la única clave de la relación {cod_edif}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {cod_Edif} es una clave.

Relación *Despacho*: está en 3FN.



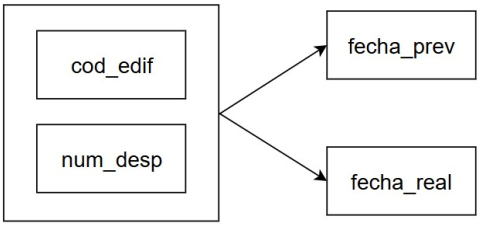
- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (orientación) depende de forma completa de la única clave de la relación {cod_edif,num_desp}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {cod_edif,num_desp} es una clave.

Relación *Despacho_nuevo*: está en 3FNG.



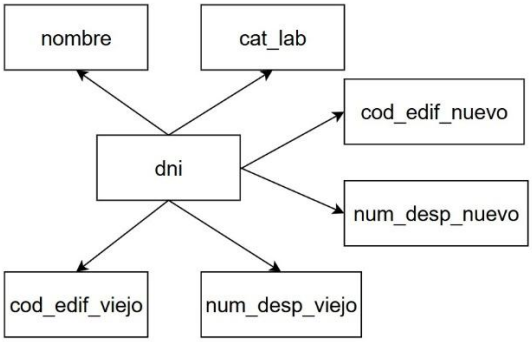
- Está en 1FN: No hay tipos estructurados.
- Está en 2FNG y 3FNG: no hay dependencias funcionales.

Relación *Despacho_viejo*: está en 3FN.



- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (fecha_prev_fecha_real) dependen de forma completa de la única clave de la relación {cod_edif,num_desp}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {cod_edif,num_desp} es una clave.

Relación *Profesor*: está en 3FN.



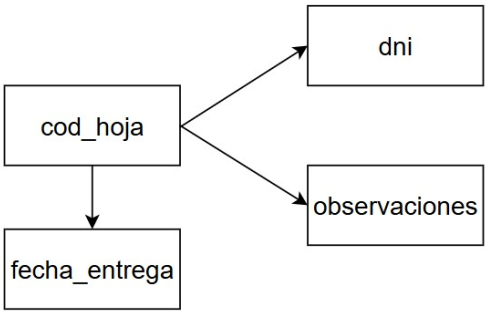
- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (nombre,cat_lab, cod_edif_nuevo, num_desp_nuevo, cod_edif_viejo, num_desp_viejo) dependen de forma completa de la única clave de la relación {dni}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {dni}. es una clave.

Relación *Contacto*: está en 3FNG.



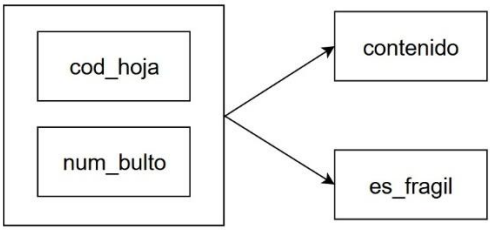
- Está en 1FN: No hay tipos estructurados.
- Está en 2FNG y 3FNG: no hay dependencias funcionales.

Relación *Hoja_traslado*: está en 3FN.



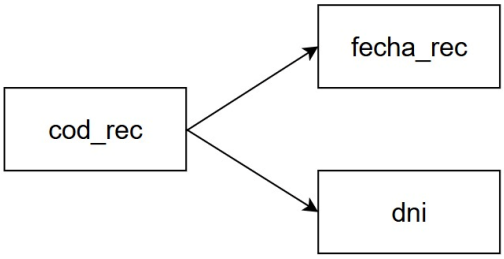
- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (dni, observaciones, fecha_entrega) dependen de forma completa de la única clave de la relación {cod_hoja}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {cod_hoja} es una clave.

Relación *Bulto*: está en 3FN.



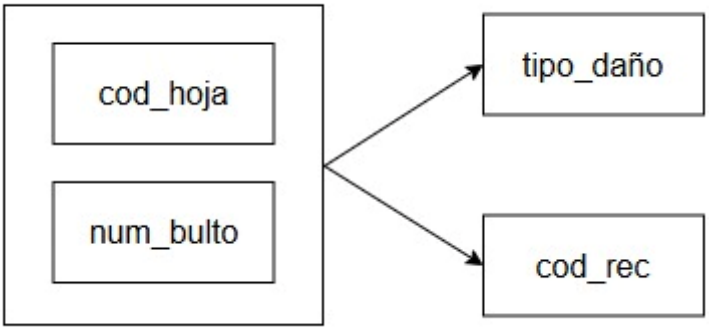
- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (contenido, es_fragil) dependen de forma completa de la única clave de la relación {cod_hoja, num_bulto}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {cod_hoja, num_bulto} es una clave.

Relación *Reclamación*: está en 3FN.



- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (fecha_rec, dni) dependen de forma completa de la única clave de la relación {cod_rec}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {cod_rec} es una clave.

Relación *Se_daña*: está en 3FN.



- Está en 1FN: No hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (tipo_daño, cod_rec) depende de forma completa de la única clave de la relación {cod_hoja, num_bulto}.
- Está en 3FN: en las tres dependencias funcionales se cumple que {cod_hoja, num_bulto} es una clave.

3.4 DIRECTRICES DE RESTAURACION DE LA INTEGRIDAD REFERENCIAL

- Relación *Despacho*:
  - Clave ajena a *Edificio*: Borrado restrictivo y modificación en cascada. No se puede borrar un edificio si hay despachos de ese edificio. Si se modifica el código de un edificio, se trasladará el cambio a los despachos de ese edificio.
- Relación *Despacho_nuevo*:
  - Clave ajena a *Despacho*: Borrado en cascada y modificación en cascada. Si se borra un despacho también se borrará si este es nuevo. Si se modifica el código de un despacho, se trasladará el cambio a los despachos que se vean afectados por esa modificación.
- Relación Despacho_viejo:
  - Clave ajena a *Despacho*: Borrado en cascada y modificación en cascada. Si se borra un despacho también se borrará si este es viejo. Si se modifica el código de un despacho, se trasladará el cambio a los despachos que se vean afectados por esa modificación.
- Relación *Profesor*:
  - Clave ajena a *Despacho_nuevo*: Borrado restrictivo y modificación en cascada. No se puede borrar un despacho si hay un profesor de ese despacho. Si se modifica el código de un despacho, se trasladará el cambio al profesor de ese despacho.
  - Clave ajena a *Despacho_viejo*: Borrado restrictivo y modificación en cascada. No se puede borrar un despacho si hay profesores de ese despacho. Si se modifica el código de un despacho, se trasladará el cambio a los profesores de ese despacho.

- Relación *Contacto*:
  - Clave ajena a *Profesor*: Borrado en cascada y modificación en cascad. Si se borra un profesor también se borrarán sus teléfonos. Si se modifica el dni de un profesor, se trasladará el cambio a sus teléfonos
- Relación *Hoja_traslado*:
  - Clave ajena a *Profesor*: Borrado en cascada y modificación en cascada. . Si se borra un profesor también se borrará su hoja de traslado. Si se modifica el dni de un profesor, se trasladará el cambio a la hoja de traslado de ese profesor.
- Relación *Bulto*:
  - Clave ajena a *Hoja_traslado*: Borrado en cascada y modificación en cascada Si se borra una hoja de traslado también se borrarán los bultos que le pertenezcan. Si se modifica el código de una hoja de traslado, se trasladará el cambio al bulto de esa hoja de traslado.
- Relación *Reclamación*:
  - Clave ajena a *Profesor*: Borrado restrictivo y modificación en cascada. No se puede borrar un profesor que haya presentado una reclamación. Si se modifica el dni de un profesor, se trasladará el cambio a la hoja de reclamación de ese profesor.
- Relación Se_daña:
  - Clave ajena a *Bulto*: Borrado en cascada y modificación en cascada. Si se borra un bulto también se borrará si este resulta dañado. Si se modifica el código de la hoja de traslado o el umero de bulto de un bulto, se trasladará el cambio al bulto dañado.
  - Clave ajena a *Reclamación*: Borrado en cascada y modificación en cascada. Si se borra una reclamación también se borrarán los bultos dañados que aparecen en esa hoja. Si se modifica el código de la hoja de reclamación se trasladará el cambio al bulto dañado.

3.5 TRANSACCIONES

- Transacción 1 (T1): listar el código de hoja, número y contenido de los bultos que son frágiles.
- Transacción 2 (T2): listar el dni y nombre de los profesores que ya han realizado el traslado: será necesario consultar el despacho viejo de los profesores donde el atributo fecha_real sea un valor no nulo.
- Transacción 3 (T3): listar el código y las observaciones de las reclamaciones presentadas, ordenadas por la fecha de entrega.
- Transacción 4 (T4): listar el código de hoja y el número de los bultos que han resultado dañados por rotura.
- Transacción 5 (T5): listar el código y la situación de aquellos edificios con más de 150 metros cuadrados de superficie.
- Transacción 6 (T6): listar el código de edificio y el número de despacho de todos los despachos que se encuentran al norte.
- Transacción 7 (T7): eliminar los edificios que no tengan ningún despacho: se buscarán los edificios cuyo código no aparezca en despacho y se eliminarán de la tabla *Edificio*.
- Transacción 8 (T8): dar de alta un nuevo profesor, para ello se le asignará un despacho viejo donde aún no se ha hecho el traspaso y se le asignará un despacho nuevo libre, también se añadirá su teléfono en la tabla *Contacto*.
- Transacción 9 (T9): calcular los despachos nuevos pendientes de traslado, para ello se contará el número de profesores que todavía no han realizado el traslado, es decir aquellos que al consultar el despacho viejo el atributo fecha_real sea un valor nulo.
- Transacción 10 (T10): asignar la fecha de realización de un traslado, para ello se buscará un despacho específico de la tabla viejo y se actualizará el atributo fecha_real con la fecha proporcionada.
- Transacción 11 (T11): presentar una reclamación por la pérdida de un bulto, se buscará un profesor que no haya presentado una reclamación y se buscará su lista de bultos, se identificara el bulto dañado mediante su número y hoja de traslado y finalmente se pondrá ‘pérdida’ en el tipo de daño.

- Transacción 12 (T12): añadir un despacho nuevo con orientación al sur, este despacho primero se registrará en la tabla *Despacho*, y luego la clave en la tabla *Despacho_nuevo*.

4 TAREA 3: CREACIÓN Y CARGA DE LAS BASES DE DATOS EN ORACLE

Tarea	Contenido	Puntos	• Trabajo a realizar	Contenido de la entrega
3	Creación de la base de datos en Oracle	1 punto	• Creación de la base de datos en Oracle con el usuario Oracle del grupo. • Carga de la base datos.	Un solo documento pdf que incluya: • Resultado de las fases anteriores con las correcciones hechas por el profesor. • Instrucciones de creación de la base de datos. • Diagrama Oracle de la base de datos generado con el *Data Modeler*. • Explicación de cómo se ha realizado la carga de datos

4.1 INSTRUCCIONES DE CREACIÓN DE LA BASE DE DATOS

4.1.1 Creación de la tabla EDIFICIO:

```
CREATE TABLE EDIFICIO(
 cod_edif VARCHAR2(2) CONSTRAINT EDIFICIO_PK PRIMARY KEY,
 situacion VARCHAR2(20 BYTE) NOT NULL,
 superficie NUMBER(6, 2) NOT NULL,
 desp_nuevos_pendientes NUMBER(4,0) DEFAULT 0 NOT NULL,
 desp_viejos_pendientes NUMBER(4,0) DEFAULT 0 NOT NULL
);
```

4.1.2 Creación de la tabla DESPACHO:

```
CREATE TABLE DESPACHO(
    cod_edif VARCHAR2(2)
        CONSTRAINT DESP_EDIF_CAJ REFERENCES EDIFICIO(cod_edif)
            DEFERRABLE INITIALLY DEFERRED,
    num_desp VARCHAR2(4 BYTE),
    orientacion VARCHAR2(1) NOT NULL,
    CONSTRAINT DESP_PK PRIMARY KEY (cod_edif,num_desp),
    CONSTRAINT orientacion_ok CHECK (orientacion IN ('N','S','E','W'))
);
```

4.1.3 Creación de la tabla DESPACHO_NUEVO:

```
CREATE TABLE DESPACHO_NUEVO(
    cod_edif VARCHAR2(2),
    num_desp VARCHAR2(4 BYTE),
    CONSTRAINT DESP_NUEVO_PK PRIMARY KEY (cod_edif,num_desp),
    CONSTRAINT DESP_NUEVO_CAJ FOREIGN KEY (cod_edif,num_desp)
        REFERENCES DESPACHO(cod_edif,num_desp)
            ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED
);
```

4.1.4 Creación de la tabla DESPACHO_VIEJO:

```
CREATE TABLE DESPACHO_VIEJO(
    cod_edif VARCHAR2(2),
    num_desp VARCHAR2(4 BYTE),
    fecha_prev DATE NOT NULL,
    fecha_real DATE,
    CONSTRAINT DESP_VIEJO_PK PRIMARY KEY (cod_edif,num_desp),
    CONSTRAINT DESP_VIEJO_CAJ FOREIGN KEY (cod_edif,num_desp)
        REFERENCES DESPACHO(cod_edif,num_desp)
            ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED
);
```

4.1.5 Creación de la tabla PROFESOR:

```
CREATE TABLE PROFESOR(
    dni VARCHAR2(9) CONSTRAINT PROF_PK PRIMARY KEY,
    nombre VARCHAR2(100 BYTE) NOT NULL,
    cat_lab VARCHAR2(40 BYTE) NOT NULL,
    cod_edif_nuevo VARCHAR2(2) NOT NULL,
    num_desp_nuevo VARCHAR2(4 BYTE) NOT NULL,
    cod_edif_viejo VARCHAR2(2) NOT NULL,
    num_desp_viejo VARCHAR2(4 BYTE) NOT NULL,
    CONSTRAINT PROF_DESP_NUEVO_CAJ FOREIGN KEY
(cod_edif_nuevo,num_desp_nuevo)
        REFERENCES DESPACHO_NUEVO(cod_edif,num_desp)
            DEFERRABLE INITIALLY DEFERRED,
    CONSTRAINT PROF_DESP_VIEJO_CAJ FOREIGN KEY
(cod_edif_viejo,num_desp_viejo)
        REFERENCES DESPACHO_VIEJO(cod_edif,num_desp)
            DEFERRABLE INITIALLY DEFERRED,
    CONSTRAINT desp_nuevo_uni UNIQUE(cod_edif_nuevo,num_desp_nuevo)
);
```

4.1.6 Creación de la tabla CONTACTO:

```
CREATE TABLE CONTACTO(
    dni VARCHAR2(9) CONSTRAINT TEL_PROF_CAJ REFERENCES PROFESOR(dni)
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,
    telefono VARCHAR2(9),
    CONSTRAINT CONTACTO_PK PRIMARY KEY (dni,telefono)
);
```

4.1.7 Creación de la tabla HOJA_TRASLADO:

```
CREATE TABLE HOJA_TRASLADO(
    cod_hoja VARCHAR2(4) CONSTRAINT H_TRAS_PK PRIMARY KEY,
    fecha_entrega DATE NOT NULL,
    observaciones VARCHAR2(120 BYTE),
    dni VARCHAR2(9) NOT NULL,
    CONSTRAINT H_TRAS_PROF_CAJ FOREIGN KEY (dni)
        REFERENCES PROFESOR(dni) ON DELETE CASCADE
            DEFERRABLE INITIALLY DEFERRED,
    CONSTRAINT prof_htras_uni UNIQUE(dni)
);
```

4.1.8 Creación de la tabla BULTO:

```
CREATE TABLE BULTO(
    cod_hoja VARCHAR2(4)
        CONSTRAINT BULTO_HTRAS_CAJ REFERENCES HOJA_TRASLADO(cod_hoja)
            ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,
    num_bult VARCHAR2(4 BYTE),
    contenido VARCHAR2(100 BYTE) NOT NULL,
    es_fragil VARCHAR2(1) NOT NULL,
    CONSTRAINT BULTO_PK PRIMARY KEY (cod_hoja,num_bult),
    CONSTRAINT es_fragil_ok CHECK (es_fragil IN ('V','F'))
);
```

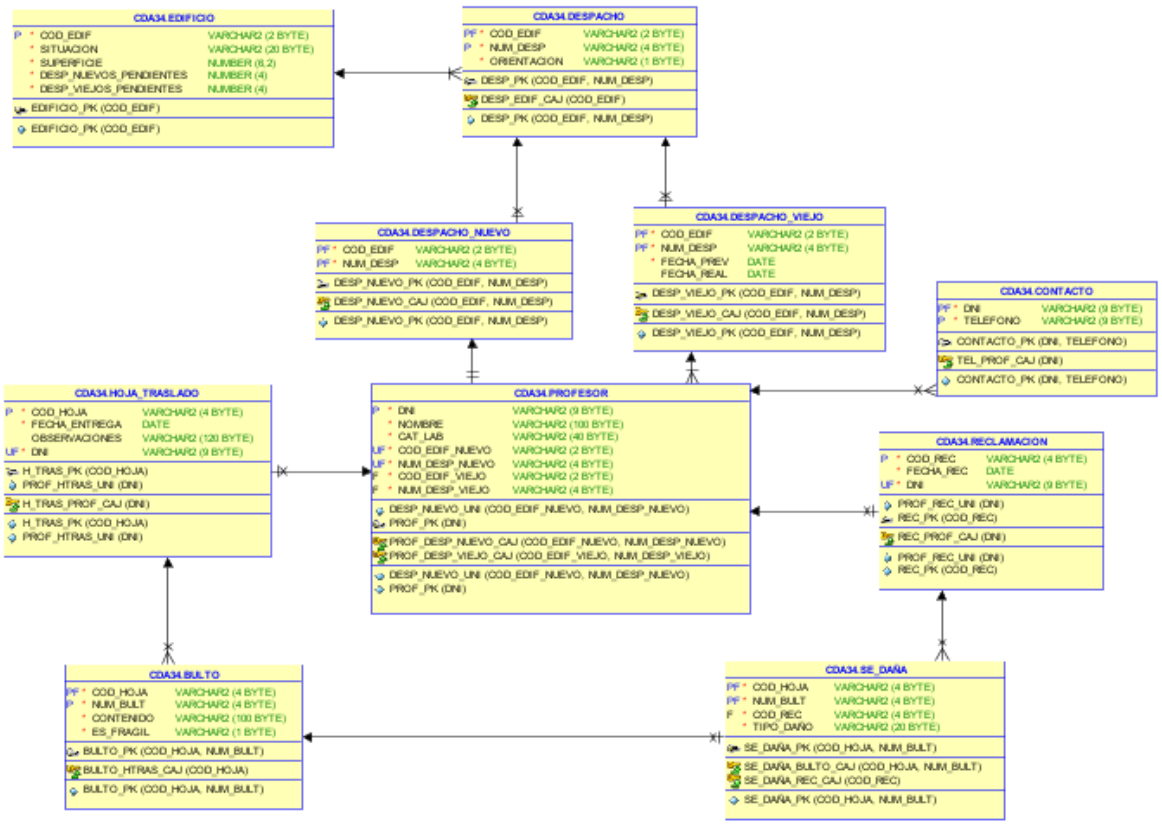
4.1.9 Creación de la tabla RECLAMACIÓN:

```
CREATE TABLE RECLAMACION(
    cod_rec VARCHAR2(4) CONSTRAINT REC_PK PRIMARY KEY,
    fecha_rec DATE NOT NULL,
    dni VARCHAR2(9) NOT NULL,
    CONSTRAINT REC_PROF_CAJ FOREIGN KEY (dni)
        REFERENCES PROFESOR(dni) ON DELETE CASCADE
            DEFERRABLE INITIALLY DEFERRED,
    CONSTRAINT prof_rec_uni UNIQUE(dni)
);
```

4.1.10 Creación de la tabla SE_DAÑA:

```
CREATE TABLE SE_DAÑA(
    cod_hoja VARCHAR2(4),
    num_bult VARCHAR2(4 BYTE),
    cod_rec VARCHAR2(4) NOT NULL,
    tipo_daño VARCHAR2(20 BYTE) NOT NULL,
    CONSTRAINT SE_DAÑA_PK PRIMARY KEY (cod_hoja,num_bult),
    CONSTRAINT SE_DAÑA_BULTO_CAJ FOREIGN KEY (cod_hoja,num_bult)
        REFERENCES BULTO(cod_hoja,num_bult)
            ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,
    CONSTRAINT SE_DAÑA_REC_CAJ FOREIGN KEY (cod_rec)
        REFERENCES RECLAMACION(cod_rec)
            ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED
);
```

4.2 DIAGRAMA GENERADO POR ORACLE CON EL DATA MODELER



4.3 CARGA DE LA BASE DE DATOS

4.3.1 Funciones de carga

4.3.1.1 Función crear_tipo_daño

```
create or replace function crear_tipo_daño
return varchar2
is
    v_random number;
    v_tipo_daño se_daña.tipo_daño%type;
begin
    v_random:=trunc(dbms_random.value(1,5));
    if v_random = 1 then v_tipo_daño := 'Rótura';
    elsif v_random = 2 then v_tipo_daño := 'Pérdida';
    elsif v_random = 3 then v_tipo_daño := 'Deformación';
    else v_tipo_daño := 'Otro';
    end if;
    return (v_tipo_daño);
end;
```

4.3.1.2 Función crear_situación

```
create or replace function crear_situacion
return varchar2
is
    v_random number;
    v_situacion varchar2(20);
begin
    v_random:=trunc(dbms_random.value(1,5));
    if v_random = 1 then v_situacion := 'Sector Norte';
    elsif v_random = 2 then v_situacion := 'Sector Este';
    elsif v_random = 3 then v_situacion := 'Sector Oeste';
    else v_situacion := 'Sector Sur';
    end if;
    return (v_situacion);
end;
```

4.3.1.3 Función crear_observaciones

```
create or replace function crear_observaciones
return varchar2
is
    v_random number;
    v_observacion hoja_traslado.observaciones%type;
begin
    v_random:=trunc(dbms_random.value(1,9));
    if v_random = 1 then v_observacion := 'Existen varios bultos con
material informático y libros antiguos que requieren manipulación
cuidadosa.';
    elsif v_random = 2 then v_observacion := 'El ordenador y el monitor
deben colocarse inmediatamente en el nuevo despacho para poder trabajar
el mismo día.';
    elsif v_random = 3 then v_observacion :=  'Documentación
confidencial: no abrir cajas y entregar directamente al profesor.';
    elsif v_random = 4 then v_observacion :=  'No apilar cajas que
contengan material electrónico.';
    elsif v_random = 5 then v_observacion :=  'El nuevo despacho se
encuentra en la segunda planta; usar ascensor.';
    elsif v_random = 6 then v_observacion :=  'Colocar las cajas junto a
la pared norte del despacho.';
    elsif v_random = 7 then v_observacion :=  'Evitar golpes y
vibraciones fuertes durante el traslado.';
    else v_observacion :=  'Avisar al profesor antes de finalizar la
descarga.';
    end if;
    return (v_observacion);
end;
```

4.3.1.4 Función crear_contenido

```
create or replace function crear_contenido
return varchar2
is
    v_random number;
    v_contenido bulto.contenido%type;
begin
    v_random :=trunc(dbms_random.value(1,7));
    if v_random = 1 then
        v_contenido := 'Impresora';
    elsif v_random = 2 then
        v_contenido := 'Caja de libros';
    elsif v_random = 3 then
        v_contenido := 'Monitor';
    elsif v_random = 4 then
        v_contenido := 'Mesa de trabajo';
    elsif v_random = 5 then
        v_contenido := 'Archivos de documentación';
    elsif v_random = 5 then
        v_contenido := 'Archivos de documentación';
    else v_contenido := 'Objetos personales';
    end if;
    return v_contenido;
end;
```

4.3.1.5 Función crear_categoria_laboral

```
create or replace function crear_categoria_laboral
return varchar2
is
    v_random number;
    v_cat_lab profesor.cat_lab%type;
begin
    v_random:=trunc(dbms_random.value(1,6));
    if v_random = 1 then v_cat_lab := 'Profesor Permanente Laboral';
    elsif v_random = 2 then v_cat_lab := 'Profesor Contratado Doctor';
    elsif v_random = 3 then v_cat_lab := 'Profesor Asociado';
    elsif v_random = 4 then v_cat_lab := 'Catedrático';
    else v_cat_lab := 'Profesor Ayudante Doctor';
    end if;
    return (v_cat_lab);
end;
```

4.3.1.6 Función contenido_frágil

```
create or replace function contenido_fragil(p_contenido
bulto.contenido%type)
return varchar2
is
    v_fragil bulto.es_fragil%type;
begin
    if p_contenido = 'Impresora' or p_contenido = 'Monitor' or
p_contenido = 'Objetos personales' then
        v_fragil := 'V';
    else v_fragil := 'F';
    end if;
    return v_fragil;
end;
```

4.3.2 Disparadores

4.3.2.1 Disparador del_desp_nuevo_pend

Este disparador resta una unidad en el edificio del que se borra un despacho nuevo pendiente de traslado.

```
create or replace trigger del_desp_nuevo_pend
after delete on profesor
for each row
declare
    v_num number;
begin
    select count(*) into v_num from despacho_viejo
        where cod_edif = :old.cod_edif_viejo and num_desp =
:old.num_desp_viejo and fecha_real is null;
    if v_num > 0 then
        update edificio
        set desp_nuevos_pendientes=desp_nuevos_pendientes-1
        where cod_edif= :old.cod_edif_nuevo;
    end if;
end;
```

4.3.2.2 Disparador ins_desp_nuevo_pend

Este disparador suma una unidad en el edificio del que se añade un despacho nuevo pendiente de traslado.

```
create or replace trigger ins_desp_nuevo_pend
after insert on profesor
for each row
declare
    v_num number;
begin
    select count(*) into v_num from despacho_viejo
        where cod_edif = :new.cod_edif_viejo and num_desp =
:new.num_desp_viejo and fecha_real is null;
    if v_num > 0 then
        update edificio
        set desp_nuevos_pendientes=desp_nuevos_pendientes+1
        where cod_edif= :new.cod_edif_nuevo;
    end if;
end;
```

4.3.2.3 Disparador del_desp_pend

Este disparador resta una unidad en el edificio del que se borra un despacho viejo pendiente de traslado.

```
create or replace trigger del_desp_pend
after delete on despacho_viejo
for each row
when(old.cod_edif is not null and old.fecha_real is null)
begin
    update edificio
    set desp_viejos_pendientes=desp_viejos_pendientes-1
    where cod_edif= :old.cod_edif;
end;
```

4.3.2.4 Disparador ins_desp _pend

Este disparador suma una unidad en el edificio del que se añade un despacho viejo pendiente de traslado.

```
create or replace trigger ins_desp__pend
after insert on despacho_viejo
for each row
when(new.cod_edif is not null and new.fecha_real is null )
begin
    update edificio
    set desp_viejos_pendientes=desp_viejos_pendientes+1
    where cod_edif= :new.cod_edif;
end;
```

4.3.3 Carga de tablas

4.3.3.1 Carga de la tabla EDIFICIO

Número de filas 9

Columnas:

- Cod_edif: los nombres de las categorías se generarán concatenando a la cadena 'E' un carácter desde el '1' al '9'.
- Situación: alfanumérica (VARCHAR2(20)). Generado con la función crear_situacion
- Superficie: NUMBER(6,2). Valor aleatorio entre 1000 y 9999.

```
declare
    v_cod_edif edificio.cod_edif%type;
    v_situacion edificio.situacion%type;
    v_superficie edificio.superficie%type;
begin
    for i in 1..9 loop
        v_cod_edif:='E'||to_char(i);
        v_situacion:= crear_situacion;
        v_superficie := round(dbms_random.value(1000,9999),2);
        insert into edificio(cod_edif,situacion,superficie)
            values(v_cod_edif,v_situacion,v_superficie);
    end loop;
    commit;
end;
```

4.3.3.2 Carga de la tabla DESPACHO

Número de filas 208

Columnas:

- Cod_edif: los nombres de las categorías se generarán concatenando a la cadena 'E' un carácter desde el '1' al '8'.
- Num_desp: los nombres de las categorías se generarán concatenando a la cadena 'D_' un carácter desde el '01' al '26'.
- Orientación: NUMBER(6,2). Valor aleatorio entre 1 y 4. Donde cada número representa N, S, E, W.

```
declare
    v_cod_edif despacho.cod_edif%type;
    v_num_desp despacho.num_desp%type;
    v_orientacion despacho.orientacion%type;
    v_random number;
begin
    for i in 1..8 loop
         v_cod_edif:='E'||to_char(i);
         for j in 1..26 loop
            if j<=9 then
                v_num_desp:='D_0'|| to_char(j);
            else v_num_desp:='D_'||to_char(j);
            end if;
            v_random := trunc(dbms_random.value(1,5));
            if v_random = 1 then v_orientacion := 'N';
            elsif v_random = 2 then v_orientacion := 'E';
            elsif v_random = 3 then v_orientacion := 'W';
            else v_orientacion := 'S';
            end if;
            insert into despacho(cod_edif,num_desp,orientacion)
                values(v_cod_edif,v_num_desp,v_orientacion);
        end loop;
    end loop;
    commit;
end;
```

4.3.3.3 Carga de la tabla DESPACHO_VIEJO

Número de filas 82

Columnas:

- Fecha_prev: Generado con la función fun.crear_fecha.
- Fecha_real: Generado con la función fun.crear_fecha.

```
declare
    cursor c_desp is select cod_edif, num_desp from despacho
                        order by num_desp fetch next 82 rows only;
    v_fecha_prev despacho_viejo.fecha_prev%type;
    v_fecha_real despacho_viejo.fecha_real%type;
    v_random number;
begin
    for v_despacho in c_desp loop
        v_fecha_prev := fun.crear_fecha('01/07/2025', '01/10/2025');
        v_random := dbms_random.value(0,1);
        if v_random<=0.5 then
            insert into despacho_viejo(cod_edif,num_desp,fecha_prev)

values(v_despacho.cod_edif,v_despacho.num_desp,v_fecha_prev);
        else v_fecha_real := fun.crear_fecha('01/08/2025', '01/12/2026');
        insert into
despacho_viejo(cod_edif,num_desp,fecha_prev,fecha_real)

values(v_despacho.cod_edif,v_despacho.num_desp,v_fecha_prev,v_fecha_real)
;
        end if;
    end loop;
    commit;
end;
```

4.3.3.4 Carga de la tabla DESPACHO_NUEVO

Número de filas 126

```
declare
    cursor c_desp is select cod_edif, num_desp from despacho
                        where (cod_edif, num_desp) not in (select
cod_edif, num_desp
                                                           from
despacho_viejo);
begin
    for v_desp in c_desp loop
        insert into despacho_nuevo(cod_edif,num_desp)
                values(v_desp.cod_edif,v_desp.num_desp);
    end loop;
    commit;
end;
```

4.3.3.5 Carga de la tabla PROFESOR

Número de filas 100

Columnas:

- dni: alfanumérica (VARCHAR2(9)). Generado con la función fun.crear_dni.
- nombre: alfanumérica (VARCHAR2(100)). Generado con la función fun.crear_nombre.
- cat_lab: alfanumérica (VARCHAR2(40)). Generado con la función fun.categoria_laboral.

```
declare
    v_dni profesor.dni%type;
    v_nombre profesor.nombre%type;
    v_cat_lab profesor.cat_lab%type;
    v_cod_edif_nuevo profesor.cod_edif_nuevo%type;
    v_num_desp_nuevo profesor.num_desp_nuevo%type;
    v_cod_edif_viejo profesor.cod_edif_viejo%type;
    v_num_desp_viejo profesor.num_desp_viejo%type;
    v_num_antiguos number;
    v_random number;
begin
    for i in 1..100 loop
        v_dni := fun.crear_dni;
        v_nombre := fun.crear_nombre;
        v_cat_lab := crear_categoria_laboral;
        select count(*)
        into v_num_antiguos
        from despacho_viejo;
        if i<=v_num_antiguos then
            select cod_edif, num_desp
            into v_cod_edif_viejo,v_num_desp_viejo
            from despacho_viejo
            offset (i-1) rows
            fetch next 1 rows only;
        else v_random:= trunc(dbms_random.value(1,v_num_antiguos));
        select cod_edif, num_desp
            into v_cod_edif_viejo,v_num_desp_viejo
            from despacho_viejo
            offset v_random rows
            fetch next 1 rows only;
        end if;
        select cod_edif, num_desp
            into v_cod_edif_nuevo,v_num_desp_nuevo
            from despacho_nuevo
            offset (i-1) rows
            fetch next 1 rows only;
        insert into
profesor(dni,nombre,cat_lab,cod_edif_nuevo,num_desp_nuevo,cod_edif_viejo,
num_desp_viejo)

values(v_dni,v_nombre,v_cat_lab,v_cod_edif_nuevo,v_num_desp_nuevo,v_cod_e
dif_viejo,v_num_desp_viejo);
    end loop;
    commit;
end;
```

4.3.3.6 Carga de la tabla HOJA_TRASLADO

Número de filas: valor aleatorio entre número de profesores y número de traslados realizados, llamado ‘k’.
Columnas:

- Cod_hoja: los nombres de las categorías se generarán concatenando a la cadena 'T_’ un carácter desde el '01' a ‘k’.
- Fecha_entrega: Generado con la función fun.crear_fecha.
- Observaciones: alfanumérica (VARCHAR2(120)). Generado con la función crear_observaciones.

```
declare
    v_cod_hoja hoja_traslado.cod_hoja%type;
    v_fecha_entrega hoja_traslado.fecha_entrega%type;
    v_observaciones hoja_traslado.observaciones%type;
    v_dni hoja_traslado.dni%type;
    v_cuantos_trasladados number;
    v_fecha_prev despacho_viejo.fecha_prev%type;
    v_fecha_real despacho_viejo.fecha_real%type;
    v_trasp_realizados number;
    v_num_prof number;
    i number;
begin
    select count(*) into v_num_prof from profesor;
    select count(*) into v_trasp_realizados
        from profesor
        where (cod_edif_viejo,num_desp_viejo) in (select cod_edif,num_desp
from despacho_viejo where fecha_real is not null);
    v_cuantos_trasladados :=
trunc(dbms_random.value(v_trasp_realizados,v_num_prof));
    i := 1;
    loop
        if i<=9 then v_cod_hoja := 'T00'||to_char(i);
        elsif i<=99 then v_cod_hoja := 'T0'||to_char(i);
        else v_cod_hoja := 'T'||to_char(i);
        end if;
        select p.dni, dv.fecha_prev, dv.fecha_real
            into v_dni, v_fecha_prev, v_fecha_real
            from profesor p, despacho_viejo dv
            where dv.cod_edif = p.cod_edif_viejo and dv.num_desp =
p.num_desp_viejo
            order by dv.fecha_real,p.dni
            offset (i-1) rows
        fetch next 1 rows only;
        if v_fecha_prev < v_fecha_real or v_fecha_real is null then
            v_fecha_entrega := fun.crear_fecha('01/05/2025', v_fecha_prev);
        else v_fecha_entrega := fun.crear_fecha('01/05/2025', v_fecha_real);
        end if;
        v_observaciones := crear_observaciones;
        insert into hoja_traslado(cod_hoja,fecha_entrega,observaciones,dni)
            values(v_cod_hoja,v_fecha_entrega,v_observaciones,v_dni);
        i := i+1;
        exit when i>v_cuantos_trasladados;
    end loop;
    commit;
end;
```

4.3.3.7 Carga de la tabla BULTO

Número de filas: valor aleatorio entre 3 y 9 bultos por hoja de traslado, llamado ‘k’.

Columnas:

- Num_bulto: los nombres de las categorías se generarán concatenando a la cadena 'B_’ un carácter desde el '01' a ‘k’ de cada hoja.
- Contenido: alfanumérica (VARCHAR2(100)). Generado con la función crear_contenido.
- Es_fragil: alfanumérica (VARCHAR2(1)). Generado con la función contenido_fragil.

```
declare
    v_cod_hoja bulto.cod_hoja%type;
    v_num_bulto bulto.num_bult%type;
    v_contenido bulto.contenido%type;
    v_fragil bulto.es_fragil%type;
    v_cuantos number;
```

```
begin
    for v_hoja in (select cod_hoja from hoja_traslado) loop
        v_cod_hoja := v_hoja.cod_hoja;
        v_cuantos := trunc(dbms_random.value(3,9));
        for i in 1..v_cuantos loop
            v_num_bulto := 'B_0'||to_char(i);
            if i=1 then
                v_contenido := 'Ordenador';
                v_fragil := 'V';
            else
            v_contenido:= crear_contenido;
            v_fragil := contenido_fragil(v_contenido);
            end if;
            insert into bulto(cod_hoja,num_bult,contenido,es_fragil)
                values(v_cod_hoja,v_num_bulto,v_contenido,v_fragil);
        end loop;
    end loop;
    commit;
end;
```

4.3.3.8 Carga de la tabla RECLAMACIÓN

Número de filas: valor aleatorio entre uno y el número de traslados realizados totales, llamado 'k'.
Columnas:

- Cod_rec: los nombres de las categorías se generarán concatenando a la cadena 'R_' un carácter desde el '01' a 'k'.
- Fecha_rec: Generado con la función fun.crear_fecha

```
declare
    v_cod_rec reclamacion.cod_rec%type;
    v_fecha_rec reclamacion.fecha_rec%type;
    v_dni reclamacion.dni%type;
    v_fecha_real date;
    v_cuantas_rec number;
    v_random number;
begin
    select count(*)
    into v_cuantas_rec
    from profesor p, despacho_viejo dv
    where p.cod_edif_viejo = dv.cod_edif and p.num_desp_viejo = dv.num_desp
and fecha_real is not null;
    v_random := trunc(dbms_random.value(1,v_cuantas_rec));
    for i in 1..v_random loop
        if i<=9 then v_cod_rec:='R00'||to_char(i);
        elsif i<=99 then v_cod_rec:='R0'||to_char(i);
        else v_cod_rec:='R'||to_char(i);
        end if;
        select p.dni, dv.fecha_real
            into v_dni,v_fecha_real
            from profesor p, despacho_viejo dv
            where p.cod_edif_viejo = dv.cod_edif and p.num_desp_viejo =
dv.num_desp and fecha_real is not null
            order by p.dni
            offset (i-1) rows
        fetch next 1 rows only;
        v_fecha_rec := fun.crear_fecha(v_fecha_real, '01/01/2026');
        insert into reclamacion(cod_rec, fecha_rec, dni)
            values(v_cod_rec,v_fecha_rec,v_dni);
    end loop;
    commit;
end;
```

4.3.3.9 Carga de la tabla SE_DAÑA

Número de filas: Por cada reclamación hay entre 1 y el número de bultos del profesor que ha presentado la reclamación.
Columnas:

- Tipo_daño: alfanumérica (VARCHAR2(20)). Generado con la función crear_tipo_daño.

```
declare
    v_cod_hoja se_daña.cod_hoja%type;
    v_num_bulto se_daña.num_bult%type;
    v_cod_rec se_daña.cod_rec%type;
    v_tipo_daño se_daña.tipo_daño%type;
    v_cuantas_rec number;
    v_cuantos_bultos number;
    v_random number;
    v_dni reclamacion.dni%type;
begin
    select count(*)
    into v_cuantas_rec
    from reclamacion;
    for i in 1..v_cuantas_rec loop
        select cod_rec, dni
            into v_cod_rec, v_dni
            from reclamacion
            order by cod_rec
            offset (i-1) rows
        fetch next 1 rows only;
        select count(*)
            into v_cuantos_bultos
            from hoja_traslado h, bulto b
            where h.dni=v_dni and b.cod_hoja=h.cod_hoja;
        v_random := trunc(dbms_random.value(1,v_cuantos_bultos+1));
        for j in 1..v_random loop
            select b.cod_hoja, b.num_bult
                into v_cod_hoja,v_num_bulto
                from hoja_traslado h, bulto b
                where h.dni=v_dni and b.cod_hoja=h.cod_hoja
                offset (j-1) rows
            fetch next 1 rows only;
            v_tipo_daño := crear_tipo_daño;
            insert into se_daña(cod_hoja,num_bult,cod_rec,tipo_daño)
                values(v_cod_hoja,v_num_bulto,v_cod_rec,v_tipo_daño);
        end loop;
    end loop;
    commit;
end;
```

4.3.3.10 Carga de la tabla CONTACTO

Número de filas: Por cada profesor hay entre 1 y 4 teléfonos.
Columnas:

- Teléfono: alfanumérica (VARCHAR2(9)). Generado con la función fun.crear_teléfono.

```
declare
    cursor c_prof is select dni from profesor;
    v_telefono contacto.telefono%type;
    v_cuantos number;
begin
    for v_prof in c_prof loop
        v_cuantos := trunc(dbms_random.value(1,4));
        for i in 1..v_cuantos loop
            v_telefono:=fun.crear_telefono;
            insert into contacto(dni,telefono)
                values(v_prof.dni, v_telefono);
        end loop;
    end loop;
    commit;
end;
```

5. TAREA 4: INFORMES Y PROGRAMACIÓN

Tarea	Contenido	Puntos	• Trabajo a realizar	Contenido de la entrega
4	Informes	0,75 puntos	• Diseñar al menos 6 informes de complejidad variada usando en BIRT (para realizar esta tarea deben haberse cargado datos en la base de datos de Oracle).	La entrega final consta de dos partes: • Memoria: un solo ocumento pdf "de calidad" que incluye: ▪ Todas las entregas realizadas en las tareas anteriores con las correcciones hechas por el profesor. ▪ Descripción de los informes generados y pantallazo de todo o de parte del cada informe. ▪ Descripción y código de los módulos programados. • Un fichero *.rptdesign* por cada informe diseñado.
	Módulos programados	0,75 puntos	Con el usuario Oracle del grupo de trabajo, escribir al menos: ▪ 3 procedimientos ▪ 3 funciones ▪ 3 disparadores El código debe ser original, no se aceptan copias de "Alquiler de Vehículos"	
	Memoria final	0,25 punto	• Redactar y maquetar documento final	

5.1 INFORMES

Los informes a desarrollar son los siguientes:

1. Listado de los bultos de cada profesor divididos por si son frágiles o no.
2. Listado de los edificios con sus datos y que muestre en un gráfico tarta cuantos despachos tiene en cada orientación.
3. Listado de todos los profesores donde aparecen sus datos y todos sus números de teléfono.
4. Elegida una hoja de reclamación, se muestra la información del profesor que la presentó, el código de la hoja de traslado y los bultos que han resultado dañados.
5. Histograma por edificios que muestra cuántos despachos nuevos pendientes de traslado y cuantos despachos viejos pendientes de traslado tienen cada uno de ellos.
6. Elegido un profesor se muestra como es la hoja de traslado presentada a secretaría.

5.1.1 INFORME 1

Listado de bultos de los profesores

Ruiz Gómez, Lucía	57443923C	Catedrático	E1 D_12
T078	Colocar las cajas junto a la pared norte del despacho.		15 may 2025, 0:00

Bultos frágiles		Bultos NO frágiles	
Número de bulto	Contenido	Número de bulto	Contenido
B_01	Ordenador	B_02	Mesa de trabajo
B_03	Impresora		
B_04	Monitor		
B_05	Objetos personales		

Baroja Sanchis, Salma	49359122H	Profesor Ayudante Doctor	E1 D_13
T071	Colocar las cajas junto a la pared norte del despacho.		25 may 2025, 0:00

Bultos frágiles		Bultos NO frágiles	
Número de bulto	Contenido	Número de bulto	Contenido
B_01	Ordenador	B_05	Caja de libros
B_02	Objetos personales		
B_03	Monitor		
B_04	Objetos personales		
B_06	Monitor		

Domínguez Sanchis, Evelio	62701721H	Profesor Contratado Doctor	E1 D_15
T021	Documentación confidencial: no abrir cajas y entregar		1 jun 2025, 0:00

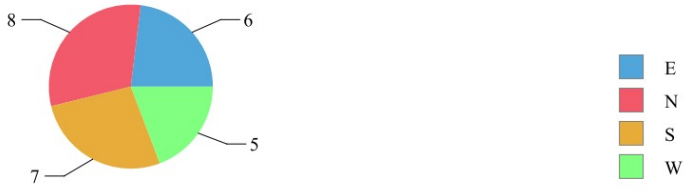
5.1.2 INFORME 2

Edificios con la distinta orientación de sus despachos

Código Edificio	Situación	Superficie
E1	Sector Este	1404,66



Código Edificio	Situación	Superficie
E2	Sector Oeste	6092,97



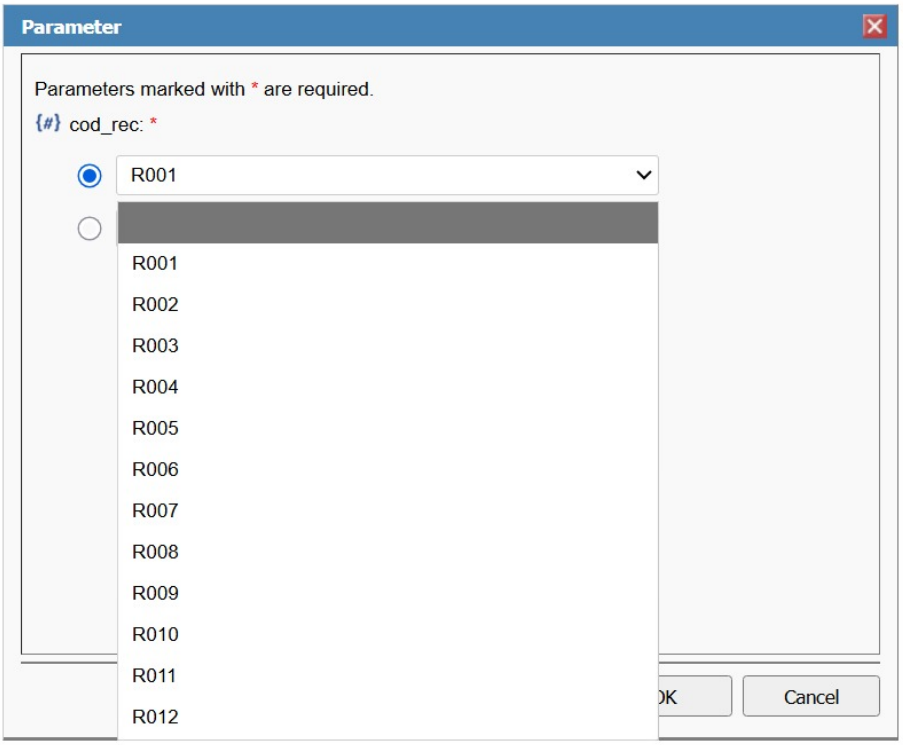
Código Edificio	Situación	Superficie

5.1.3 INFORME 3

Número de teléfono de los profesores

Nombre	Dni	Categoria Laboral	Teléfonos
Casamayor Pérez, Omar	10484540J	Catedrático	
			448701043
			775595817
			807728452
		Teléfenos totales:	3
Baroja Álvarez, Cruz	10802761Y	Profesor Permanente Laboral	
			703708397
			940165953
		Teléfenos totales:	2
Fernández Martínez, Celia	11398901D	Profesor Permanente Laboral	
			312847084
			835821027
			996449492
		Teléfenos totales:	3
Herranz Rodríguez, Carmen	11818956S	Catedrático	
			151402866
			467897903
			700093261
		Teléfenos totales:	3
Sanchis Sánchez, Laura	12095213L	Profesor Ayudante Doctor	

5.1.4 INFORME 4

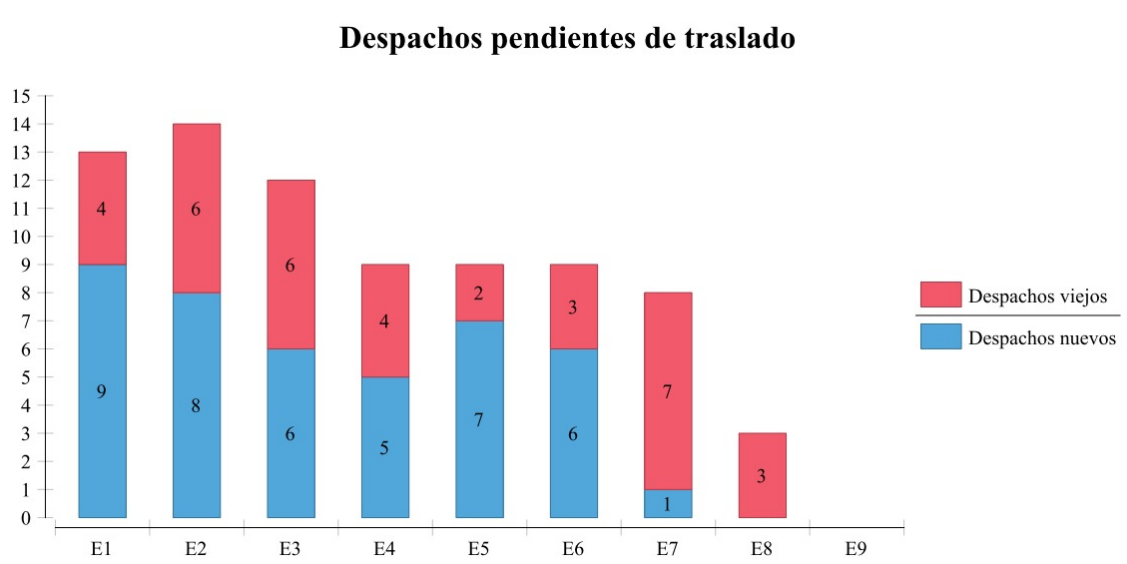


Estos bultos dañados son de la siguiente hoja de Reclamación: **R001**

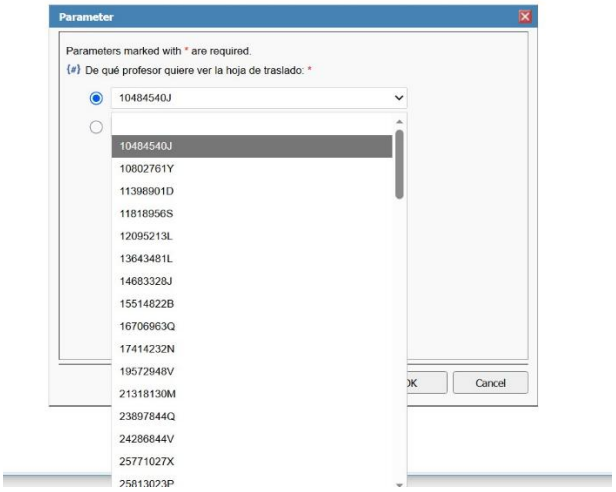
Profesor	Dni	Categoria Laboral	Despacho		Fecha de Reclamación
Fernández Martínez, Celia	11398901D	Profesor Permanente Laboral	E3	D_25	10 dic 2025, 0:00

Hoja de Traslado	Bulto	Daño sufrido	Contenido	Es frágil?
T009				
	B_01	Deformación	Ordenador	V
	B_02	Otro	Caja de libros	F
			El número de bultos dañados es:	2

5.1.5 INFORME 5



5.1.6 INFORME 6



Hoja de traslado

DNI:	10484540J
Nombre:	Casamayor Pérez, Omar
Categoría Laboral:	Catedrático
Teléfono:	448701043 775595817 807728452

Código de hoja	Observaciones	Fecha de entrega	Despacho nuevo		Despacho Viejo	
T057	Avisar al profesor antes de finalizar la descarga.	12 sept 2025, 0:00	E2	D_15	E2	D_08

Lista de bultos:

Número de bulto	Contenido	Es frágil?
B_01	Ordenador	V
B_02	Impresora	V
B_03	Monitor	V
B_04	Mesa de trabajo	F
B_05	Archivos de documentación	F
B_06	Caja de libros	F
B_07	Monitor	V
B_08	Mesa de trabajo	F

5.2 PROGRAMACION

5.2.1 FUNCIONES

Además de las funciones creadas para la carga de datos hemos creado 4 más:

5.2.1.1 FUNCION RECLAMACION

Se pasa un dni a una función y esta devuelve si el profesor ha presentado una reclamación o no.

```
create or replace function es_bulto_de(p_cod_hoja bulto.cod_hoja%type,
p_num_bulto bulto.num_bult%type, p_dni profesor.dni%type)
return varchar2
is
    v_cod_hoja bulto.cod_hoja%type;
    v_existe number;
    v_existe2 number;
begin
    select count(*) into v_existe2 from hoja_traslado where dni=p_dni;
    if v_existe2 < 1 then return 'F';
    end if;
    select cod_hoja into v_cod_hoja from hoja_traslado where dni=p_dni;
    if p_cod_hoja <>  v_cod_hoja then
        return 'F';
    end if;
    select count(*) into v_existe from bulto where cod_hoja=p_cod_hoja
         and num_bult=p_num_bulto;
    if v_existe <1 then return 'F';
    else return 'V';
    end if;
end;
```

5.2.1.2 FUNCION BULTO CORRESPONDIENTE

Se pasa un dni, un código de hoja de traslado y un número de bulto a una función y esta devuelve si el bulto corresponde al profesor.

```
create or replace function tiene_reclamacion(p_dni profesor.dni%type)
return varchar2
is
    v_existe number;
begin
    select count(*) into v_existe from reclamacion where dni=p_dni;
    if v_existe <1 then return 'F';
    else return 'V';
    end if;
end;
```

5.2.1.3 FUNCION DESPACHOS NUEVOS PENDIENTES

Devuelve el número total de despachos nuevos pendientes de traslado.

```
create or replace function despN_totales_pend
return number
is
    v_resp number;
begin
    select sum(desp_nuevos_pendientes) into v_resp from edificio;
    return v_resp;
end;
```

5.2.1.4 FUNCION DESPACHOS VIEJOS PENDIENTES

Devuelve el número total de despachos viejos pendientes de traslado.

```
create or replace function despV_totales_pend
return number
is
    v_resp number;
begin
    select sum(desp_viejos_pendientes) into v_resp from edificio;
    return v_resp;
end;
```

5.2.2 PROCEDIMIENTOS

5.2.2.1 PROCEDIMIENTO ELIMINAR EDIFICIOS

Este procedimiento resuelve la transacción T7 donde se eliminan los edificios que no tienen ningún despacho. En este procedimiento no se incluye el commit para poder deshacer el borrado si se desea.

```
create or replace procedure borrar_edif is
begin
    delete from edificio where cod_edif not in (select distinct cod_edif
    from despacho);
end;
```

5.2.2.2 PROCEDIMIENTO AÑADIR DESPACHOS SUR

Este procedimiento resuelve la transacción T12 donde se añade un despacho situado al sur en un edificio cualquiera. En este procedimiento se incluye el commit para poder confirmar la transacción realizada.

```
create or replace procedure insertar_desp_nuevo_alsur is
    v_cod_edif despacho.cod_edif%type;
    v_num_desp despacho.num_desp%type;
    v_orient despacho.orientacion%type;
    v_cuant_ed number;
    v_cuant_desp number;
    v_rand number;
begin
    select count(*) into v_cuant_ed from edificio;
    v_rand := trunc(dbms_random.value(1, v_cuant_ed+1));
    v_cod_edif := 'E'||to_char(v_rand);
    select count(*) into v_cuant_desp from despacho where
    cod_edif=v_cod_edif;
    if (v_cuant_desp+1) < 10 then
        v_num_desp := 'D_0'||to_char(v_cuant_desp+1);
    else v_num_desp := 'D_'||to_char(v_cuant_desp+1);
    end if;
    v_orient:= 'S';
    insert into despacho(cod_edif, num_desp, orientacion)
        values(v_cod_edif,v_num_desp,v_orient);
    insert into despacho_nuevo(cod_edif, num_desp)
        values(v_cod_edif,v_num_desp);
    commit;
        dbms_output.put_line( 'Se ha añadido el despacho '||v_num_desp  ||'
        al edificio: '|| v_cod_edif );
end;
```

5.2.2.3 PROCEDIMIENTO HOJA RECLAMACION

Este procedimiento resuelve la transacción T11 donde se presenta un hoja de reclamación por la pérdida de un bulto de un profesor que ya ha realizado el traslado. En este procedimiento se incluye el commit para poder confirmar la transacción realizada.

```
create or replace procedure present_rec(p_dni profesor.dni%type)
is
    v_dni reclamacion.dni%type; v_fecha_rec reclamacion.fecha_rec%type;
    v_cod_rec reclamacion.cod_rec%type; v_cuantas_rec number;
    v_fecha_real despacho_viejo.fecha_real%type;
    v_cod_hoja hoja_traslado.cod_hoja%type;
    v_num_bulto bulto.num_bult%type;
    v_cuant_bultos number;  v_random number;
    v_tipo_daño se_Daña.tipo_daño%type;
begin
    select count(*) into v_cuantas_rec from reclamacion;
    if (v_cuantas_rec+1)<10 then
    v_cod_rec:='R00'||to_char((v_cuantas_rec+1));
        elsif (v_cuantas_rec+1)<100 then
    v_cod_rec:='R0'||to_char((v_cuantas_rec+1));
        else v_cod_rec:='R'||to_char((v_cuantas_rec+1));
    end if;
    v_dni:=p_dni;
    select dv.fecha_real into v_fecha_real from profesor p, despacho_viejo
    dv
        where p.cod_edif_viejo=dv.cod_edif and
    p.num_desp_viejo=dv.num_desp
            and p.dni= p_dni;
    v_fecha_rec := fun.crear_fecha(v_fecha_real, '01/01/2026');
    insert into reclamacion(cod_rec, fecha_rec, dni)
            values(v_cod_rec,v_fecha_rec,v_dni);
    dbms_output.put_line( 'El profesor '||v_dni ||' ha presentado una
    reclamacion '||v_fecha_rec );
    select cod_hoja into v_cod_hoja from hoja_traslado where dni=v_dni;
    select count(*) into v_cuant_bultos from bulto where
    cod_hoja=v_cod_hoja;
    v_random :=trunc(dbms_random.value(1, v_cuant_bultos+1));
    v_num_bulto := 'B_0' || to_char(v_random);
    v_tipo_daño:= 'Pérdida';
    insert into se_daña(cod_hoja,num_bult,cod_rec,tipo_daño)
        values(v_cod_hoja,v_num_bulto,v_cod_rec,v_tipo_daño);
    commit;
end;
```

5.2.2.4 PROCEDIMIENTO PROFESORES YA TRASPASADOS

Este procedimiento resuelve la transacción T2 donde se listan el dni y el nombre de los profesores que ya han realizado su traspaso.

```
create or replace procedure prof_tras_real is
    v_cuantos number;
begin
    v_cuantos := 0;
    dbms_output.put_line('Los siguientes profesores han realizado ya su
    traspaso: ');
    for profesor in (select p.dni dni , p.nombre nombre from profesor p,
    despacho_viejo dv
                        where dv.cod_edif=p.cod_edif_viejo and
    dv.num_desp=p.num_desp_viejo and dv.fecha_real is not null) loop
        dbms_output.put_line( profesor.nombre|| '  ' ||profesor.dni );
        v_cuantos := v_cuantos +1;
    end loop;
    dbms_output.put_line( 'Lo han realizado '|| to_char(v_cuantos) ||'
    profesores.'  );
end;
```

5.2.3 DISPARADORES

Ya había 5 disparadores para mantener actualizado los atributos derivados de edificio y se han añadido otros 6 para controlar las restricciones de integridad que no se podían expresar:

5.2.3.1 DISPARADOR BULTO DAÑADO

El disparador regula que, cuando se presenta un bulto dañado, este bulto pertenezca al profesor que ha presentado la reclamación.

```
create or replace trigger disp_control_bulto_reclamado
after insert on se_daña
for each row
declare
    v_dni_reclamacion reclamacion.dni%type;
    v_dni_hoja hoja_traslado.dni%type;
    v_existe number;
begin
    select dni
    into v_dni_reclamacion
    from reclamacion
    where cod_rec = :new.cod_rec;

    select dni
    into v_dni_hoja
    from hoja_traslado
    where cod_hoja = :new.cod_hoja;

    if v_dni_reclamacion <> v_dni_hoja then
        raise_application_error(-20006,
            'El bulto reclamado no pertenece al profesor que presenta la
    reclamación');
    end if;
    select count(*)
    into v_existe
    from bulto
    where cod_hoja= :new.cod_hoja and num_bult= :new.num_bult;
    if v_existe <1 then
        raise_application_error(-20007,'El bulto reclamado no existe');
    end if;
end ;
```

5.2.3.2 DISPARADOR DESPACHO NUEVO/VIEJO

Regula que un despacho solo pueda ser o nuevo o viejo y no ambas a la vez.

```
create or replace trigger disp_control_desp_nuevo
after insert on despacho_nuevo
for each row
declare
    v_existe number;
begin
    select count(*)
    into v_existe
    from despacho_viejo
    where cod_edif= :new.cod_edif and num_desp= :new.num_desp;
    if v_existe > 0 then
        raise_application_error(-20005,
        'Ya existe ese despacho viejo, por lo tanto no puede ser nuevo');
    end if;
end;
```

5.2.3.3 DISPARADOR DESPACHO NUEVO/VIEJO

También regula que un despacho solo pueda ser o nuevo o viejo y no ambas a la vez.

```
create or replace trigger disp_control_desp_viejo
after insert on despacho_viejo
for each row
declare
    v_existe number;
begin
    select count(*)
    into v_existe
    from despacho_nuevo
    where cod_edif= :new.cod_edif and num_desp= :new.num_desp;
    if v_existe > 0 then
        raise_application_error(-20005,
        'Ya existe ese despacho nuevo, por lo tanto no puede ser viejo');
    end if;
end;
```

5.2.3.4 DISPARADOR FECHA RECLAMACION

Regula que la fecha de la reclamación sea posterior a cuando se realiza el traspaso.

```
create or replace trigger disp_control_fecha_daño
after insert on reclamacion
for each row
declare
    v_fecha_real date;
begin
    select dv.fecha_real
    into v_fecha_real
    from profesor p, despacho_viejo dv
    where p.dni = :new.dni and p.cod_edif_viejo = dv.cod_edif and
    p.num_desp_viejo=dv.num_desp;

    if v_fecha_real is null then
        raise_application_error(-20004,
        'Si el traspaso aún no ha sido trasladado no se puede registrar un
    daño');
    end if;

    if :new.fecha_rec < v_fecha_real then
        raise_application_error(-20005,
        'La fecha de reclamación debe ser posterior a la fecha de
    realización del traslado');
    end if;
end;
```

5.2.3.5 DISPARADOR FECHA PREVISTA Y REALIZADA

Regula que tanto la fecha prevista como la fecha realizada sean posteriores a la fecha de entrega de la hoja de traslado.

```
create or replace trigger disp_control_fechas
after insert or update of fecha_real,fecha_prev on despacho_viejo
for each row
declare
    v_fecha_hoja hoja_traslado.fecha_entrega%type;
    v_fecha_eleg date;
begin
    select max(h.fecha_entrega)
    into v_fecha_hoja
    from profesor p, hoja_traslado h
    where p.dni=h.dni and p.cod_edif_viejo= :new.cod_edif and
    p.num_desp_viejo= :new.num_desp;

    if (:new.fecha_real is null) or :new.fecha_prev <= :new.fecha_real
    then
        v_fecha_eleg:= :new.fecha_prev;
    else v_fecha_eleg:= :new.fecha_real;
    end if;
    if v_fecha_hoja > v_fecha_eleg then
        raise_application_error(-20005,'La fechas no cuadran');
 end
```

5.2.3.6 DISPARADOR RESTAR VIEJOS PENDIENTES

Cuando se añade una fecha de traslado ya realizado, este disparador automáticamente resta 1 en los despachos viejos pendientes. También resta cuantos profesores han realizado el traspaso en el número de despachos nuevos que quedan por hacer el traslado.

```
create or replace trigger upd_desp_pend
after update on despacho_viejo
for each row
when(new.cod_edif is not null and new.fecha_real is not null )
begin
    update edificio
    set desp_viejos_pendientes=desp_viejos_pendientes-1
    where cod_edif= :new.cod_edif;
    for c_edif in (select cod_edif_nuevo, count(*) cuenta from profesor
                   where cod_edif_viejo= :new.cod_edif and
    num_desp_viejo= :new.num_desp group by cod_edif_nuevo) loop
        update edificio
        set desp_nuevos_pendientes = desp_nuevos_pendientes- c_edif.cuenta
        where cod_edif=c_edif.cod_edif_nuevo;
    end loop;
end;
```